

# Roteiro da Aula 2: banco, ActiveRecord e o model `User`

---

**Deck:** `slides/build/aula-02-banco-de-dados-e-activerecord.pptx` (54 slides, sendo 7 de prática, que cobrem as 8 da apostila) **Apostila da turma:** `02-activerecord.md` · **Checkpoint:** `aula-02` **Antes de tudo:** `guia-do-instrutor.md` — conduzir a sala, não o conteúdo

É a aula mais tranquila das quatro em termos de tempo. Use a folga para o console ao vivo — é o que mais fixa.

---

## Antes de começar

- [ ] Gabarito em `~/capacitacao-gabarito`, branch `aula-02`, com o banco já preparado.
- [ ] `docker compose up -d` rodando no seu, para o console não travar na hora.
- [ ] Terminal grande, com o `bin/rails console` já aberto e testado.
- [ ] Ter em mãos um exemplo real de vazamento de senha para citar (LinkedIn 2012, 6,5 milhões de hashes SHA-1 sem salt — quebrados em dias).
- [ ] Conferir quem não terminou a prática da Aula 1 e resolver nos primeiros 10 minutos.

---

## Cronograma

**As oito práticas são o esqueleto do dia**, uma por bloco: explica, faz, explica, faz. Estão em negrito, e quando atrasar você corta **conteúdo**, nunca prática.

| Relógio      | Slides    | Bloco                                                                  | Min |
|--------------|-----------|------------------------------------------------------------------------|-----|
| 00:00        | 1–3       | Abertura e retomada                                                    | 6   |
| 00:06        | 4–9       | Banco, transação, Postgres, container                                  | 18  |
| 00:24        | <b>10</b> | <b>Prática 1 — o banco de pé, e o bcrypt</b>                           | 5   |
| 00:29        | 11–20     | ActiveRecord, <code>blank?</code> , migrations, <code>schema.rb</code> | 30  |
| 00:59        | <b>21</b> | <b>Prática 2 — as três migrations</b>                                  | 20  |
| <b>01:19</b> | —         | <b>Intervalo</b>                                                       | 10  |
| 01:29        | 22–27     | Senha: hash, bcrypt, <code>has_secure_password</code>                  | 18  |
| 01:47        | <b>28</b> | <b>Prática 3 — o bcrypt no console</b>                                 | 10  |
| 01:57        | 29–32     | Validações e normalização                                              | 14  |
| 02:11        | <b>33</b> | <b>Práticas 4 e 5 — o validador e o model</b>                          | 35  |
| 02:46        | 34–40     | Associações e N+1                                                      | 20  |
| 03:06        | <b>41</b> | <b>Prática 6 — a segunda tabela</b>                                    | 15  |
| 03:21        | 42–45     | Concerns                                                               | 14  |
| 03:35        | <b>46</b> | <b>Prática 7 — os dois concerns</b>                                    | 20  |
| 03:55        | 47–51     | Console ao vivo, seeds, fixtures, teste                                | 15  |
| 04:10        | <b>52</b> | <b>Prática 8 — fixtures, testes e commit</b>                           | 25  |
| 04:35        | 53–54     | Recapitulação e fim                                                    | 4   |

**Dá 4h39.** É a aula em que eles mais escrevem código, e o tempo de digitação é real. Corte **nesta ordem**:

| # | O que cortar                                                                                             | Ganho |
|---|----------------------------------------------------------------------------------------------------------|-------|
| 1 | Prática 8 vira dever de casa: em aula, só as fixtures e <code>bin/rails test</code> rodando              | –15   |
| 2 | Slide 34 ( <code>O MODELO RELACIONAL</code> , <code># CORTÁVEL</code> ) se a turma já viu SQL            | –5    |
| 3 | Prática 3 (bcrypt no console) vira demo sua, projetada, em 4 min                                         | –6    |
| 4 | Slide 40 ( <code>A ARMADILHA N+1</code> ) — fica na apostila e volta na Aula 3                           | –5    |
| 5 | Práticas 4 e 5: dê o validador pronto (do gabarito) e eles só escrevem o <code>User</code>               | –15   |
| 6 | Prática 6: você faz projetado, eles copiam a migration                                                   | –8    |
| 7 | Prática 7: dê o <code>EmailVerifiable</code> pronto e eles escrevem só o <code>PasswordResettable</code> | –8    |

Cortando de 1 a 4, fecha em **3h48**. Cortando os sete, **3h02**.

**Não corte a Prática 2.** Migration mal feita é o que trava a Aula 3 inteira, e o erro só aparece uma semana depois.

## Bloco a bloco

### 00:00 · Retomada (1–3)

O slide 3 é a ponte: *"temos uma API que responde. Ela não guarda nada. Reiniciou, esqueceu tudo. Hoje ela ganha memória."*

### 00:06 · Banco (4–9)

**Transação (6–7) é o slide que eles vão precisar na Aula 3.** Use o exemplo do dinheiro: debitar de um e creditar no outro têm que ser a mesma operação; debitar sozinho é dinheiro que sumiu.

Aponte o `!` no slide 7 e explique: dentro da transação você **quer** que estoure, porque `save` devolve `false` em silêncio e a transação seguiria feliz gravando metade.

No 8, a frase: *"desenvolver no mesmo banco da produção elimina uma categoria inteira de bug."*

### 00:29 · ActiveRecord (11–16)

O slide 13 é o que causa espanto em quem vem de Django: **a classe não declara nada.**

```
class User < ApplicationRecord
end
```

Diga: *"isso já tem todas as colunas. A fonte da verdade é a migration, não a classe."*

**15 é para rodar no console**, não para ler:

```
nil.blank?      # true
"".blank?       # true
" ".blank?      # true
0.blank?        # false    ← aqui alguém vai reclamar
```

E a pegadinha que vem de Python: em Ruby, `if 0` executa. `if ""` executa. Só `nil` e `false` são falsos.

### 00:44 · Migrations (17–20)

Gere uma migration ao vivo:

```
bin/rails generate migration CreateUsers
```

Mostre o arquivo criado, o timestamp no nome, e diga que é ele que define a ordem.

**O slide 18 tem o ponto mais importante do bloco:** índice único é restrição do banco, validação é mensagem bonita. Duas requisições simultâneas passam pela validação juntas — as duas consultam, as duas não acham ninguém — mas só uma vence o índice.

No 19, a regra: **nunca edite migration que já rodou em produção**. Crie outra.

## 01:29 · Senha (22–27)

O bloco mais importante da aula inteira.

- **21** — comece pelo susto. Cite o vazamento que você separou.
- **22** — hash não é criptografia. Criptografia tem volta; hash não.
- **23** — por que bcrypt e não SHA-256: SHA é rápido, **e é esse o problema**. GPU testa bilhões por segundo. bcrypt é lento de propósito, com custo ajustável.
- **24** — desmonte o hash na tela: versão, custo, salt, digest.

Demonstre no console — é o momento em que a ficha cai:

```
BCrypt::Password.create("senha123")
BCrypt::Password.create("senha123")    # rode DE NOVO
```

**São diferentes.** É o salt. Pergunte por que, antes de responder.

## 01:57 · Validações (29–32)

O 29 (normalizar antes de validar) fecha com o 18: se você não normaliza, o índice único não serve para nada, porque o banco acha que `Ana@UFOP.br` e `ana@ufop.br` são valores diferentes.

## 02:46 · Associações (34–40)

Bloco novo, e é o que eles mais vão usar no primeiro projeto de verdade.

A regra que resume tudo (slide 32): **a chave estrangeira fica no lado "muitos"**. Quem carrega a coluna usa `belongs_to`; quem é apontado usa `has_many`.

No console:

```
ana = User.first
ana.login_events.create!(client: "mobile", occurred_at: Time.current)
ana.login_events.count
ana.login_events.recentes.first.user.name    # e volta
```

**Demonstre o N+1 de verdade** (36) — com o log na tela:

```
User.all.each { |u| puts u.login_events.count }    # conte as consultas
User.includes(:login_events).each { |u| puts u.login_events.count }
```

A diferença no log é o argumento. Nenhum slide convence tanto.

### 03:21 · Concerns (42–45)

A ponte com a Aula 1: *"lembram do módulo que se inclui numa classe? Isto aqui é ele, com nome de Rails."*

O slide 40 tem as três decisões — a que mais rende é a terceira: `email_verified_at` é data, não booleano, porque um dia alguém vai abrir chamado perguntando *quando* a conta foi ativada.

### 03:55 · Console e testes (47–51)

O slide 51 (`authenticate_by_email`) merece atenção: o ataque de temporização. Se a resposta volta em 1ms quando o e-mail não existe e em 100ms quando existe, dá para descobrir quem tem conta cronometrando. Guarde — volta na Aula 3.

## Conduzindo as oito práticas

Cada prática tem, na apostila, a lista de comandos, uma conferência e uma **tabela de erros**. Mande abrir a apostila na prática correspondente — não dite os comandos.

| #     | Slide | O que cobrar em voz alta                                                                              |
|-------|-------|-------------------------------------------------------------------------------------------------------|
| 1     | 10    | O <code>docker compose ps</code> tem que dizer <b>healthy</b> , não <code>starting</code>             |
| 2     | 21    | Uma migration por vez, e <code>db:migrate</code> entre elas. A conferência é o <b>16</b>              |
| 3     | 28    | Rodar o <code>create</code> duas vezes e ver dar diferente. Isso é o <i>salt</i>                      |
| 4 e 5 | 33    | Quando <code>valid?</code> der <code>false</code> , o reflexo é <code>p u.errors.full_messages</code> |
| 6     | 41    | O avançado (apagar o usuário e ver os eventos sumirem) vale fazer ao vivo                             |
| 7     | 46    | O banco guarda o <b>digest</b> do código, não o código                                                |
| 8     | 52    | <code>users(:ana)</code> , nunca <code>User.first</code> , dentro de teste                            |

Os pontos onde a turma trava, em ordem de frequência:

- esqueceram `db:migrate` depois de criar a migration;
- escreveram `has_secure_password` sem adicionar a gem `bcrypt` no Gemfile;
- `matricula` com máscara ( `20.112-34` ) falhando na validação — é exatamente o motivo de `normalize_matricula` existir;
- `PG::DuplicateTable` de quem rodou a migration duas vezes: `db:drop db:create db:migrate`;
- teste que passa sozinho e falha em conjunto: é `User.first` dentro do teste.

Quando uma migration falhar, a resposta está na **linha seguinte** da mensagem, não na primeira. Ensine isso uma vez e economize meia hora.

## Perguntas que vão aparecer

| Pergunta                                                          | Resposta curta                                                                                                                                                                                                                 |
|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Por que não SQLite? É mais simples."                             | Uma escrita por vez no banco inteiro. Num servidor com várias requisições, trava.                                                                                                                                              |
| "Dá para descriptografar o <code>password_digest</code> ?"        | Não. Não é criptografia, é hash: caminho só de ida. Nem você consegue ver a senha do usuário — e isso é a intenção.                                                                                                            |
| "Por que o bcrypt é lento? Isso não é ruim?"                      | É lento <b>de propósito</b> . 100ms para você é nada; para quem tenta bilhões de senhas, é o que inviabiliza.                                                                                                                  |
| "Se eu já valido no model, para que o índice único?"              | Concorrência. Duas requisições ao mesmo tempo passam pelas duas validações e só uma vence o índice.                                                                                                                            |
| "Por que não <code>t.boolean :email_verified?</code> "            | Porque booleano responde "sim" e data responde "sim, dia 12 às 14h". Um dia alguém vai perguntar quando.                                                                                                                       |
| "Posso apagar uma migration antiga e refazer?"                    | Na sua máquina, sim. Depois que rodou em produção, nunca — crie outra.                                                                                                                                                         |
| " <code>dependent: :delete_all</code> ou <code>:destroy</code> ?" | <code>delete_all</code> é um <code>DELETE</code> só, direto no banco, e não roda callback. <code>:destroy</code> carrega cada registro e roda os callbacks. Aqui não há callback, então <code>delete_all</code> é mais rápido. |

## Dever de casa

1. Terminar as práticas que não couberam.
2. Bônus: escrever o N+1 de propósito, contar as consultas no log e consertar com `includes`.
3. Ler a seção "Uma sutileza de segurança" da apostila — ela é o começo da próxima aula.